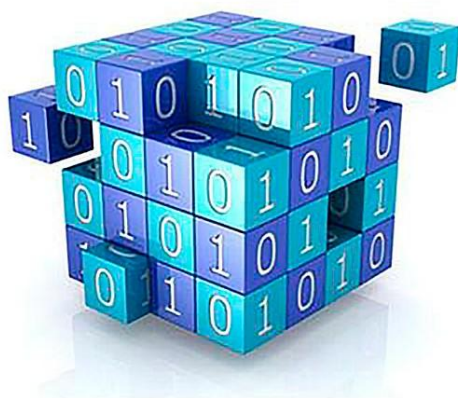




ESTRUCTURA DE DATOS

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:

Manejo de Estructura de
Datos

Unidad #: 3
Semana 14

Fecha: 26-05-2025

Temas: Recursividad: Funciones Recursivas y Recursión vs
Iteración

Autor: _____

Introducción

Bienvenidos al material de lectura de la semana #14 de la Unidad #3 de la materia Manejo de Estructura de Datos. En el presente material veremos y explicaremos sobre Recursividad: Funciones Recursivas y Recursión vs Iteración.

Este material de lectura está diseñado para proporcionar una base sólida sobre los fundamentos de programación y análisis de algoritmos, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

¡Antes de comenzar, asegúrate de tener papel y lápiz a la mano para realizar las actividades interactivas!

Sección 1: Recursividad

La recursividad es una técnica de programación en la que una función o algoritmo se llama así mismo para resolver un problema.

Existen algunas características de los algoritmos recursivos:

- Deben existir una salida en la que no se haga la llamada recursiva (caso base).
- La llamada recursiva debe ser una versión más simple que la llamada que la invoco.

La recursividad se utiliza comúnmente para recorrer y manipular estructuras de datos jerárquicas o recursivas, como árboles y listas enlazadas.

Aunque la recursividad permite expresar soluciones simples y naturales a ciertos problemas, también tiene sus ventajas y desventajas.

Ventajas:

- Permite expresar soluciones simples y naturales a ciertos tipos de problemas.
- Puede reducir la complejidad y el tiempo necesario para escribir y entender el código.
- Facilita la implementación de algoritmos que se pueden dividir en subproblemas similares, como por ejemplo los de ordenación, búsqueda y recorrido de estructuras de datos.
- La eficiencia de la recursividad reside en el hecho de que se puede usar para resolver problemas de difícil solución iterativa.
- No es necesario definir la secuencia de pasos exacta para resolver el problema.
- Agrega claridad y reduce el tiempo necesario para escribir y entender el código, ya que la recursividad permite expresar algoritmos de manera más concisa y elegante.

Desventajas:

- Son menos eficientes en términos de tiempo de ejecución debido a las operaciones auxiliares y llamadas adicionales a funciones que conlleva.
- Es necesaria la creación de varias variables locales en cada llamada recursiva, lo que puede ocasionar problemas de memoria.
- Sobrecarga asociada con las llamadas a subfunciones recursivas, lo que puede ralentizar la ejecución del algoritmo.
- Ocupan más memoria debido a las llamadas recursivas.

En síntesis, podemos decir que la recursividad es una técnica poderosa para resolver problemas de manera elegante, especialmente aquellos relacionados con estructuras de datos jerárquicas, pero eso sí debe usarse con cuidado para evitar desbordamiento de pila.

Sección 2: Funciones recursivas.

Las funciones recursivas son un tipo de función en programación que se llamas a sí mismas para resolver un problema de manera más eficiente. Estas funciones son muy útiles para resolver problemas complejos utilizando un enfoque dividir para conquistar, y suelen ser más concisas y fáciles de entender que otras técnicas para resolver problemas similares.

Existen dos tipos de funciones recursivas: Directa e Indirecta o cruzada.

Recursión Directa.

Se presenta cuando una función invoca a sí misma directamente, como en el cálculo del factorial de un número.

Recursión Indirecta o Cruzada.

Ocurre cuando una función involucra a otra función que a su vez invoca a la primera, creando un ciclo de llamadas recursivas entre ambas funciones.

Sección 3: Recursión vs Iteración.

Solución Iterativa o Recursiva No hay problemas intrínsecamente iterativos o recursivos. Sí hay problemas para los cuales la solución iterativa o recursiva es más sencilla, más natural, más fácil de ver.

En algunos casos, el algoritmo recursivo es mucho más compacto y simple que el iterativo. **Solución recursiva:** costo de ejecución relativamente alto: Número de Llamadas, manejo de la pila. Costo en tiempo y en espacio de memoria por cada registro de activación.

Sólo se suele elegir la solución recursiva cuando: La penalización en tiempo y espacio no es significativa. La solución recursiva es más elegante, legible y/o fácil de mantener.

En programación, las operaciones pueden ser manejadas por recursividad o iteraciones. Recursividad es cuando utilizamos un método que se llama a sí mismo, directa o indirectamente. Una de las estructuras

fundamentales de programación son los bucles, las cuales forman parte de las estructuras de la mayoría de los lenguajes.

Por ejemplo, existen dos estrategias para calcular $1+2+3+\dots+n$. La primera estrategia es utilizando recursividad:

```
public static int sumaN_recursivo(int n){  
    return (n<=0)?0:n + sumaN_recursivo(n-1);  
}
```

Luego, para iteración el método sería:

```
public static int sumaN_iterativo(int n){  
    int result= 0;  
    for(int i=1; i<=n; i++){  
        result +=i;  
    }  
    return result;  
}
```

Bibliografía

- Mary E. 2015. Estructura de datos-.
- Rodriguez L. 2012. Fundamentos de programación II. Universidad Pontifica de Salamanca.